Supporting type definition in Grammar 4 Aug 2025

$D \rightarrow T\ id$

$T \rightarrow int$

$T \rightarrow float$

int x
float y
int[10][20] k

$D \rightarrow T\ id$

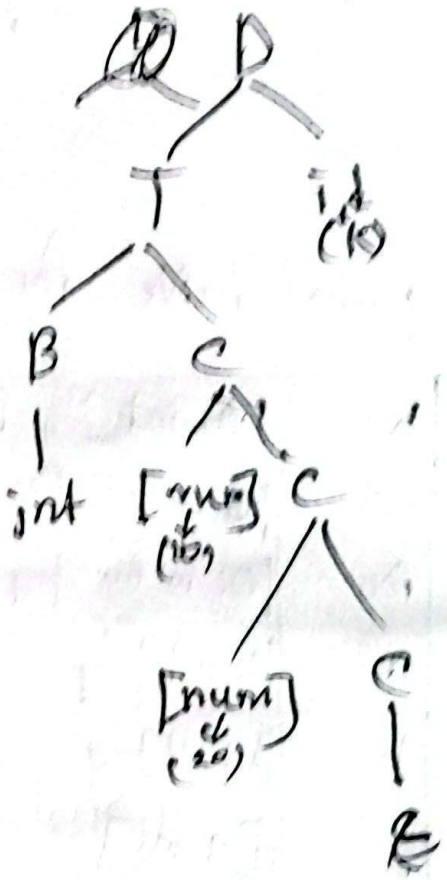
$T \rightarrow BC$

$B \rightarrow int$

$B \rightarrow float$

$C \rightarrow \epsilon$

$C \rightarrow [num]\ C$



Scanned with CamScanner

$D \rightarrow \epsilon$

$D \rightarrow T\ id\ ;\ D$

$T \rightarrow BC$

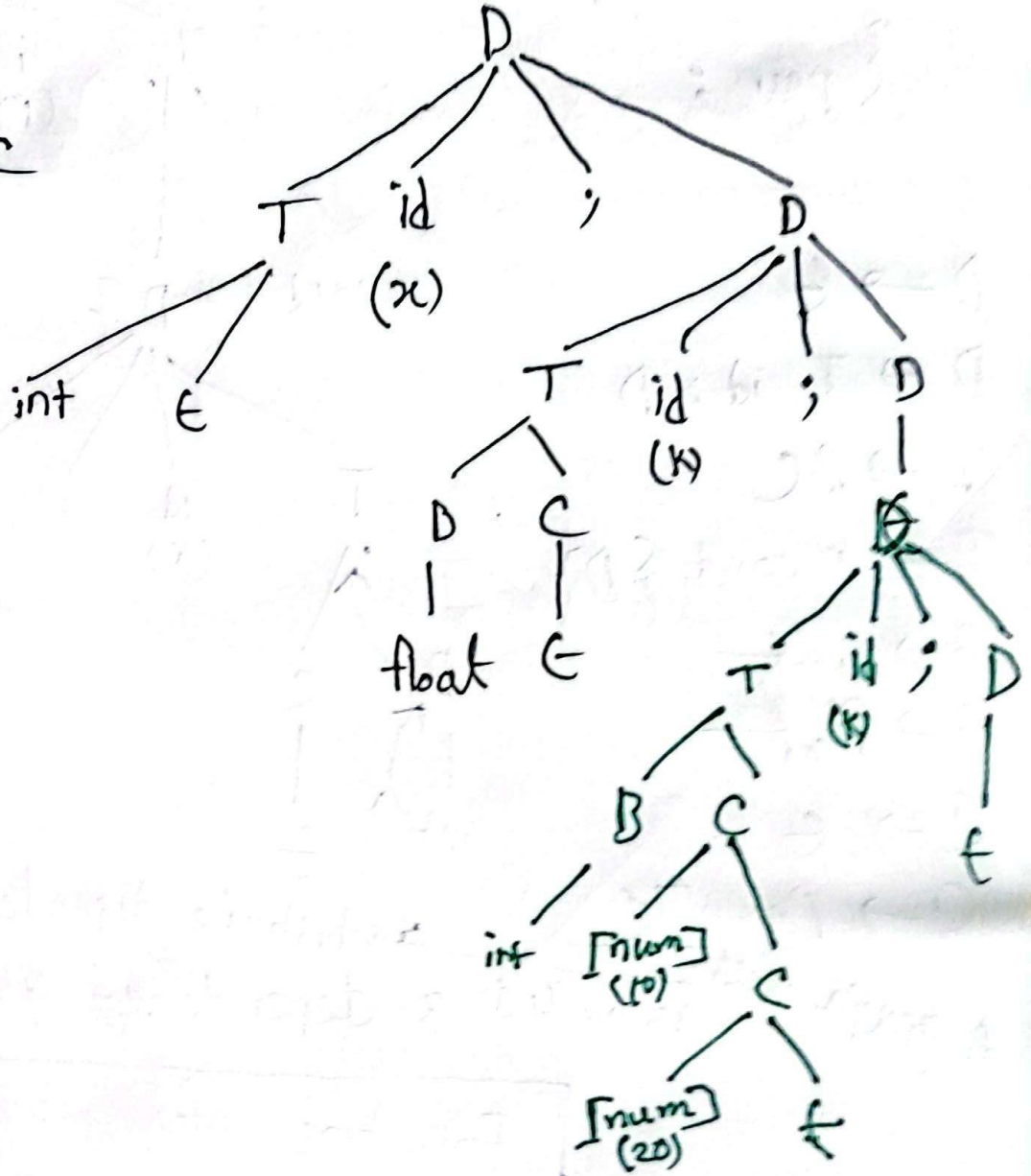
$B \rightarrow int$

$B \rightarrow float$

$C \rightarrow \epsilon$

$C \rightarrow [num]\ C$

```
int x;
float y;
int [10][20] K;
```



Healthcare

Alexa

Scanned with CamScanner

Structures in C

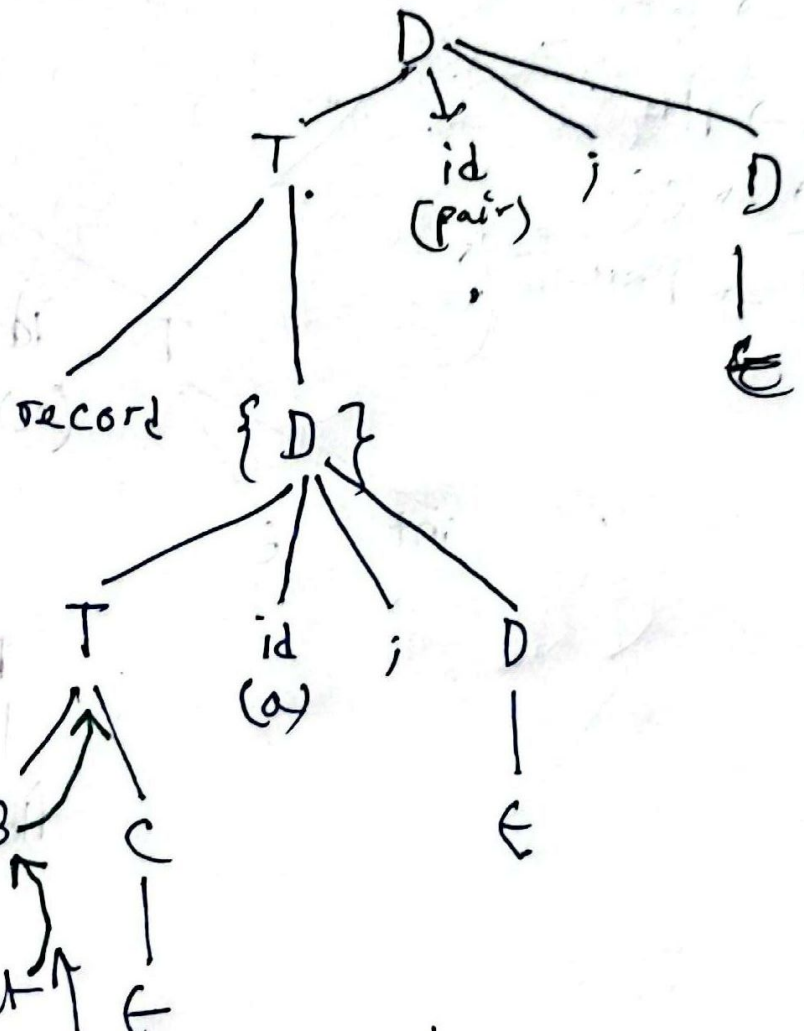
book's term → "record"

struct ~~[illegible]~~ {

statement 1 ;

statement 2 ;

} pair ;



D → ϵ

D → T id ; D

T → BC

T → record { D }

B → int

B → float

C → ϵ

C → [num] C

A graph with → attribute dependency arrows is called a dependency graph

"For type information we need an L-attributed SDT"

Scanned with CamScanner

Tree traversal

Post-order ⟶ Left child ⟶ Right child ⟶ Root

`int a[10][20], b[30][10][50];`

this is broken down into multiple statements in the compiler preprocessing step phase.

Healthcare

Alexa

Scanned with CamScanner

Writing SDT Rules

6 Aug 2025

$P \rightarrow D$

$D \rightarrow T\ id;\ \{Key = id.lexeme;\ SymbolTable.insert(Key, Store: T.t, with: T.w);\ off = off + T.w\}$

$D \rightarrow \epsilon$

$T \rightarrow B\ \{C.bt = B.t;\ C.bw = B.w;\}\ C$

$B \rightarrow int\ \{B.t = int;\ B.w = 4;\}$

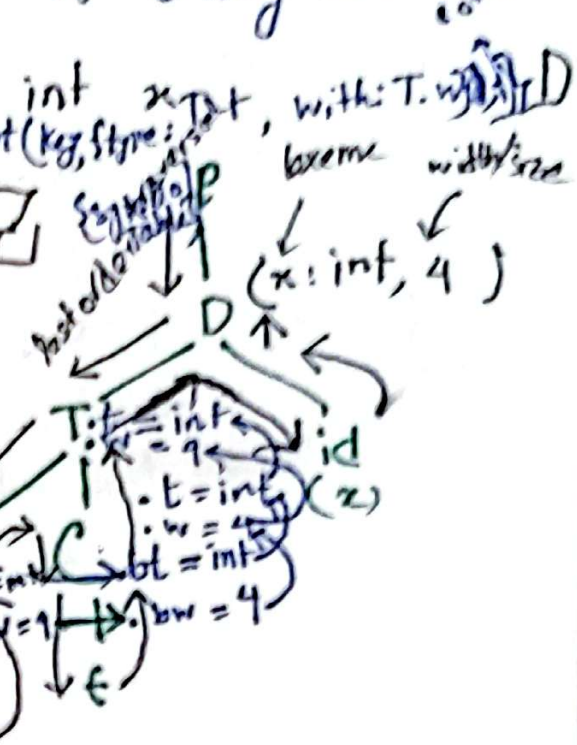
$B \rightarrow float\ \{B.t = float,\ B.w = 8\}$

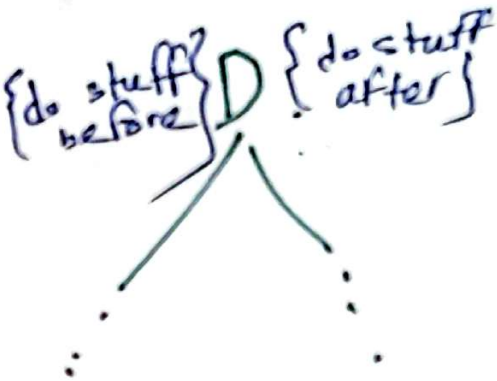
$C \rightarrow \epsilon\ \{C.t = C.bt;\ C.w = C.bw;\}$

$C \rightarrow [num]\ C_1\ \{C.t = array(num.value, C_1.t);\ C.w = C_1.w * num.value;\}$

in Bison its yylval

int x





$\{T.t = C.t;\ T.w = C.w;\}$

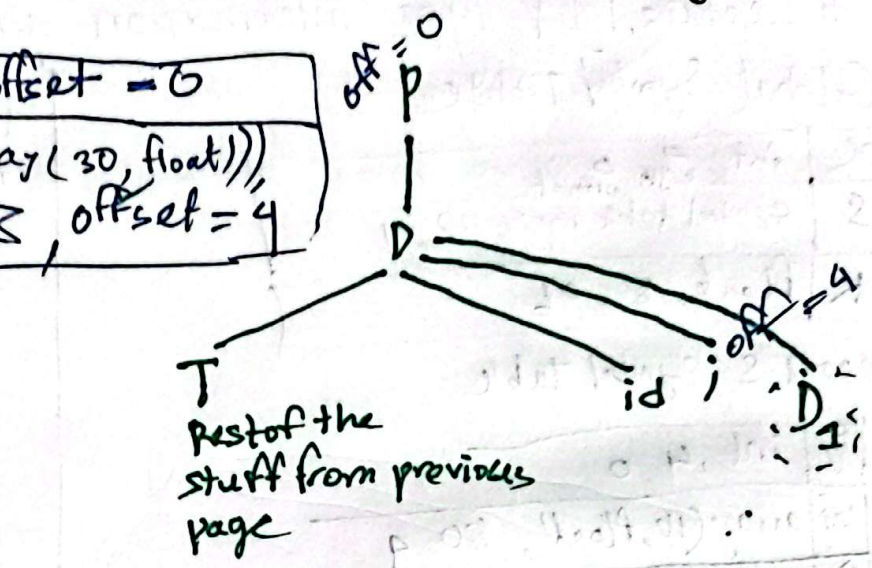
$\{SymbolTable = new\ HashTable();\ off = 0;\}$

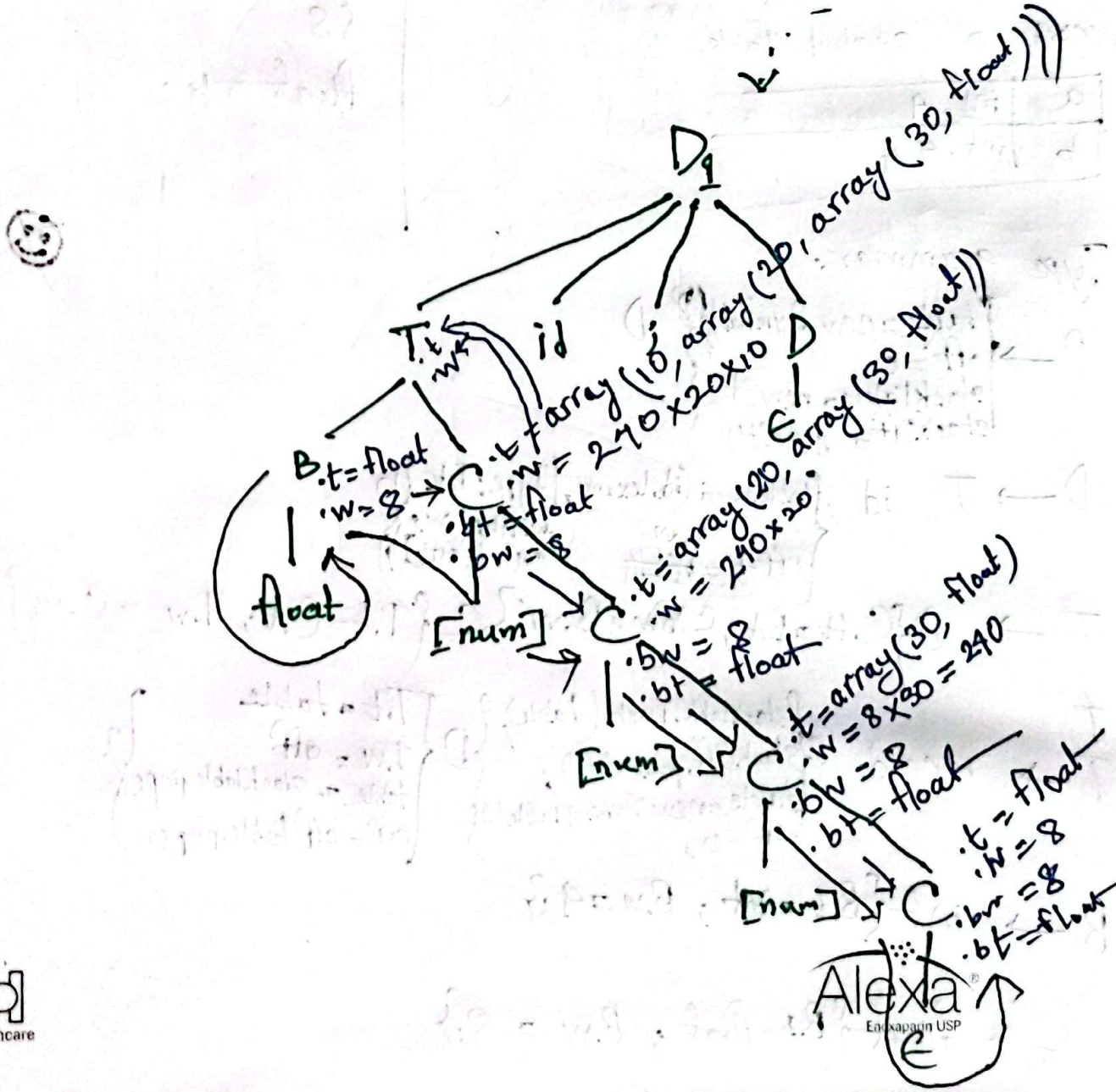
$\{C_1.bt = C.bt;\ C_1.bw = C.bw;\}$

Scanned with CamScanner

```
int x ;
float[10][20][30] y;
```

key	attributes
x	type = int, width = 4, offset = 0
y	type = array(10, array(20, array(30, float))), width = 10x20x30x8, offset = 4





Scanned with CamScanner

11 Aug 2025

Lecture 19 + Type information processing

Global Symbol Table

x	int, 4, 0
s	symbol table for s, 92, 4
k	float, 8, 96

record s : Symbol Table

y	int, 4, 0
w	array(10, float), 80, 4
p	symbol Table for p, 8, 84

record p Symbol Table

a	int, 4
b	int, 4, 4

```
int x;
record {
    int y;
    float[10] w;
    record {
        int a;
        int b;
    } p;
} s;
float k;
```

Type grammar:

$P \rightarrow$ { table = new Sym.Table(); off = 0; stackTable = new St(); stackOff = new St() } D

$D \rightarrow T$ id ; { table.insert(id.lexeme, {type: T.t, width: T.w, offset: off}); off += T.width }

$T \rightarrow B$ { C.bt = B.t; C.bw = B.w } C { T.t = C.t; T.w = C.w; }

$T \rightarrow$ record { stackTable.push(table); stackOff.push(off); table = new SymbolTable(); off = 0; } D { T.t = table; T.w = off; table = stackTable.pop(); off = off + table.pop(); }

$B \rightarrow$ int { B.t = int; B.w = 4; }

$B \rightarrow$ float { B.t = float; B.w = 8; }

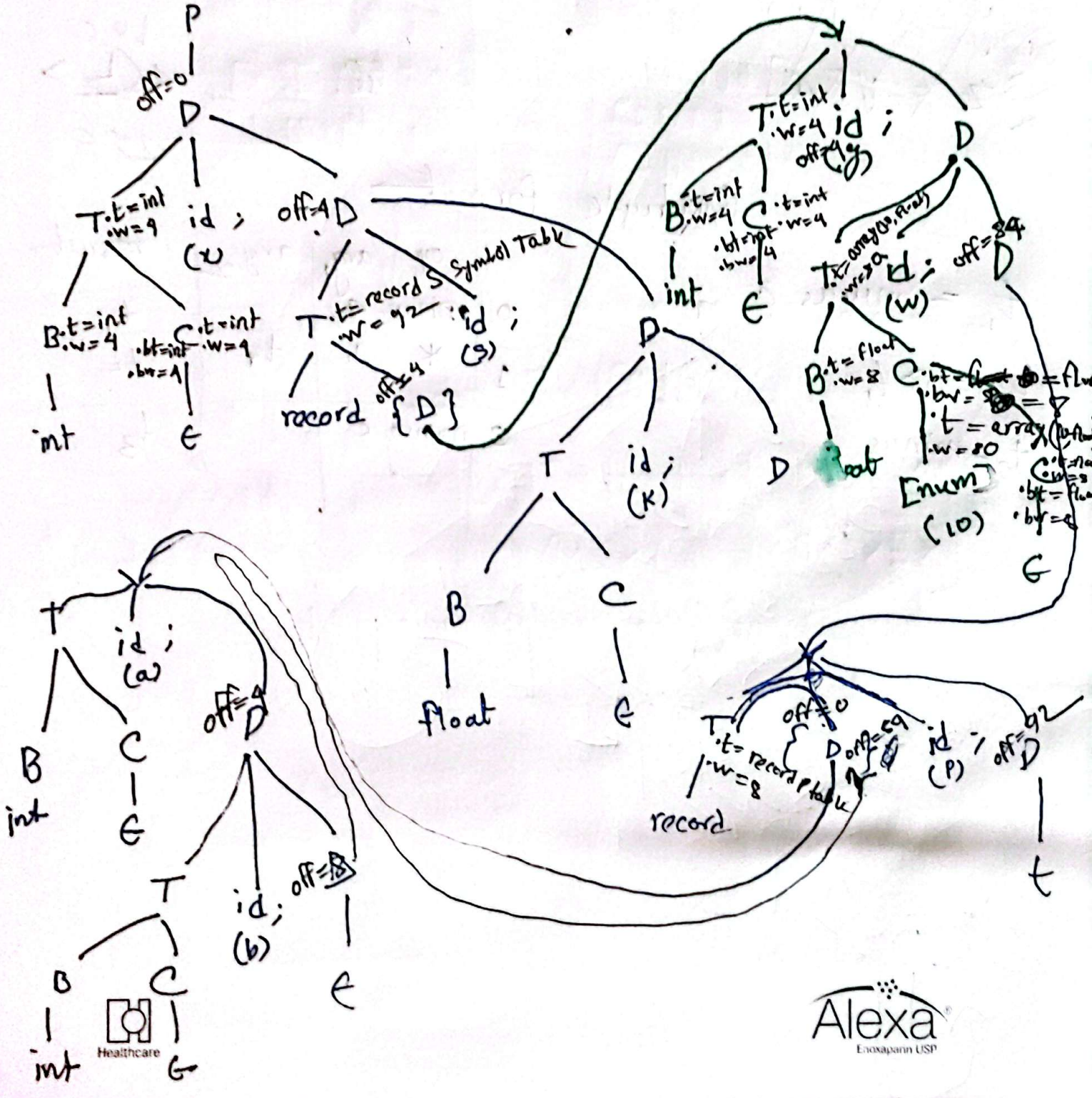
Scanned with CamScanner

$C \rightarrow [num]\{C_1.bt = C.bt; C_1.bw = C.bw;\} C_1 \{C.b = array(num.value, C_1.t); C.w = num.value * C_1.w;\}$

$bt = t = float$

$C \rightarrow \in \{C.t = C.bt ; C.w = C.bw\}$

Parse tree



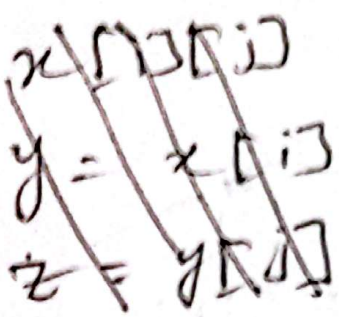
Alexa
Enoxaparin USP

Scanned with CamScanner

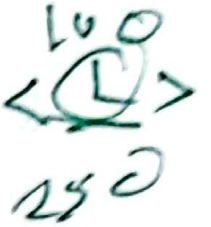
13 Aug 2025

4. Three-Address Code

→ "No more than three addresses but can be less than three"







Quadruple format →

t_1 = minus c

$t_2 = b * t_1$

t_3 = minus c

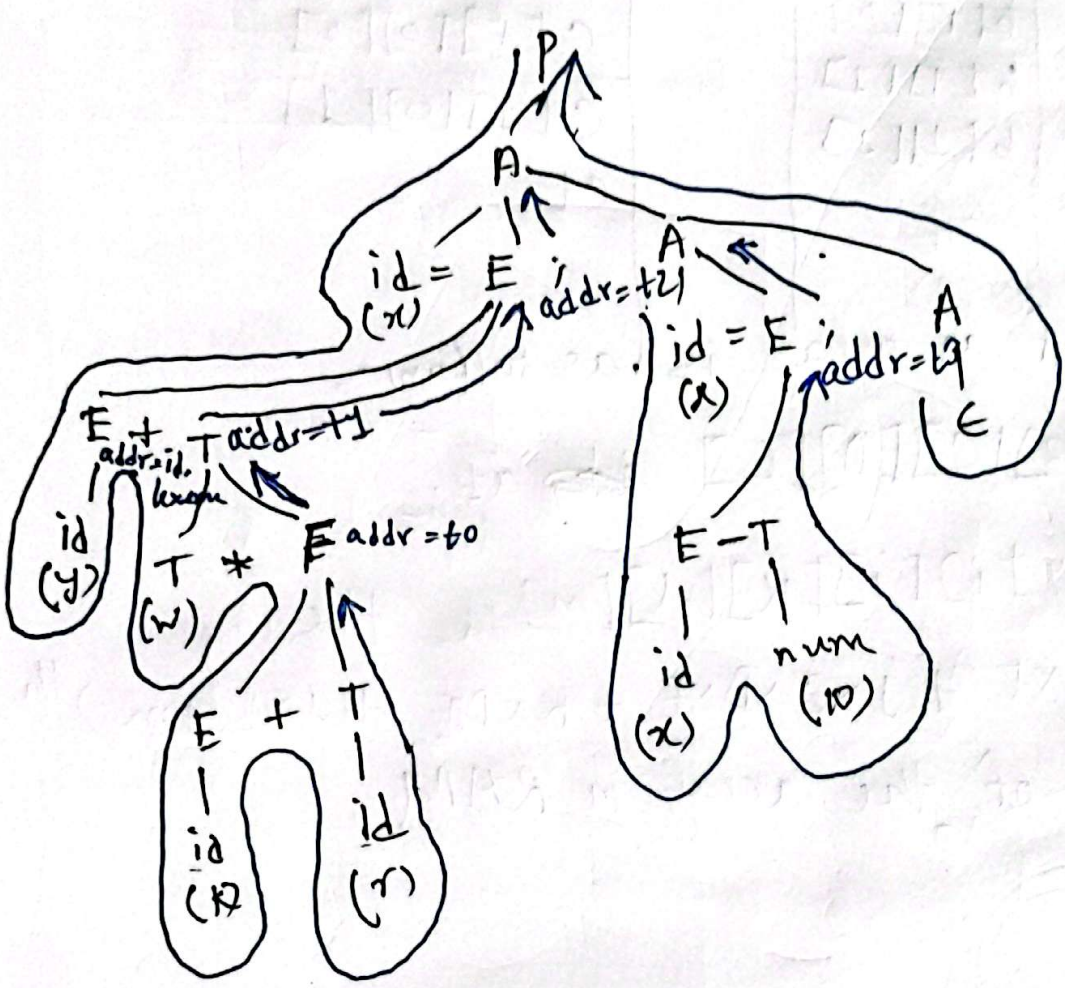
	op	arg_1	arg_2	result
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3

Scanned with CamScanner

Example

$P \rightarrow A$
$A \rightarrow \epsilon$
$A \rightarrow id = E ; A$
$E \rightarrow E+T \mid E-T \mid T$
$T \rightarrow T*F \mid F/F \mid F$
$F \rightarrow id \mid num \mid (E)$

```
x = y+w*(k+r);
z = x-10;
```



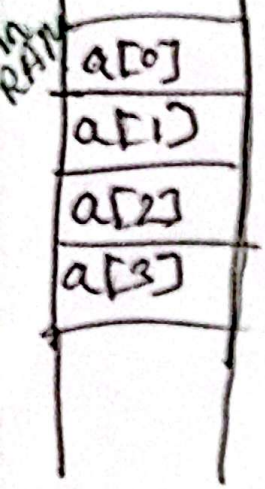

```
t0 = k+r
t1 = w*t0
t2 = y+t1
x = t2
t3 = x-10
z = t3
```

Healthcare
Alexa
Enoxaparin USP
Scanned with CamScanner

~~13 Aug 2025~~

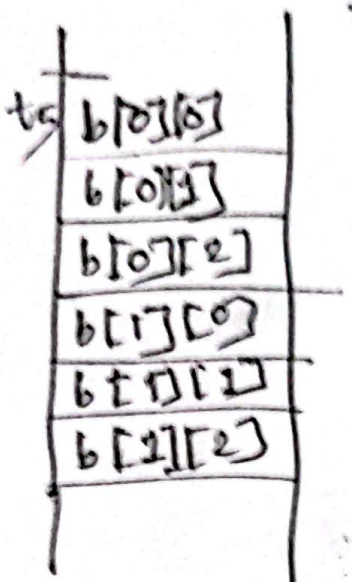
20 Aug 2025

int[4] a

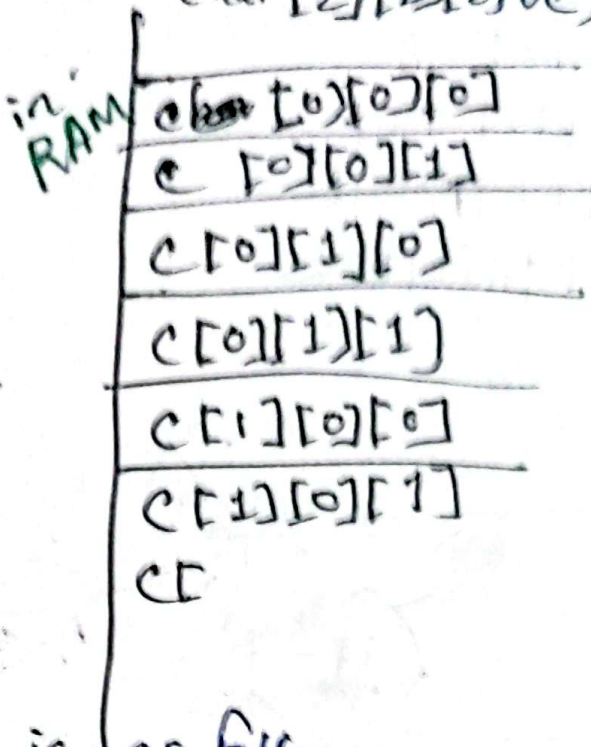


float [2][3] b;

int RAM



char [2][2][2] c;



in general if an array is as follows

int [A][B][C][D][E] a1;

then array [i][j][k][l][m] in the

$(i \times B \times C \times D \times E + j \times C \times D \times E + k \times D E + l \times E + m)^{th}$ element of the array in RAM

Scanned with CamScanner

See the PDF 'Compiler-principle-technique-tools.pdf' (Page 406/1018)

$S \rightarrow id = E;$

$\quad | \; L = E;$ (array access)

$E \rightarrow E_1 + E_2$

$\quad | \; id$

$\quad | \; L$

$L \rightarrow id[E]$

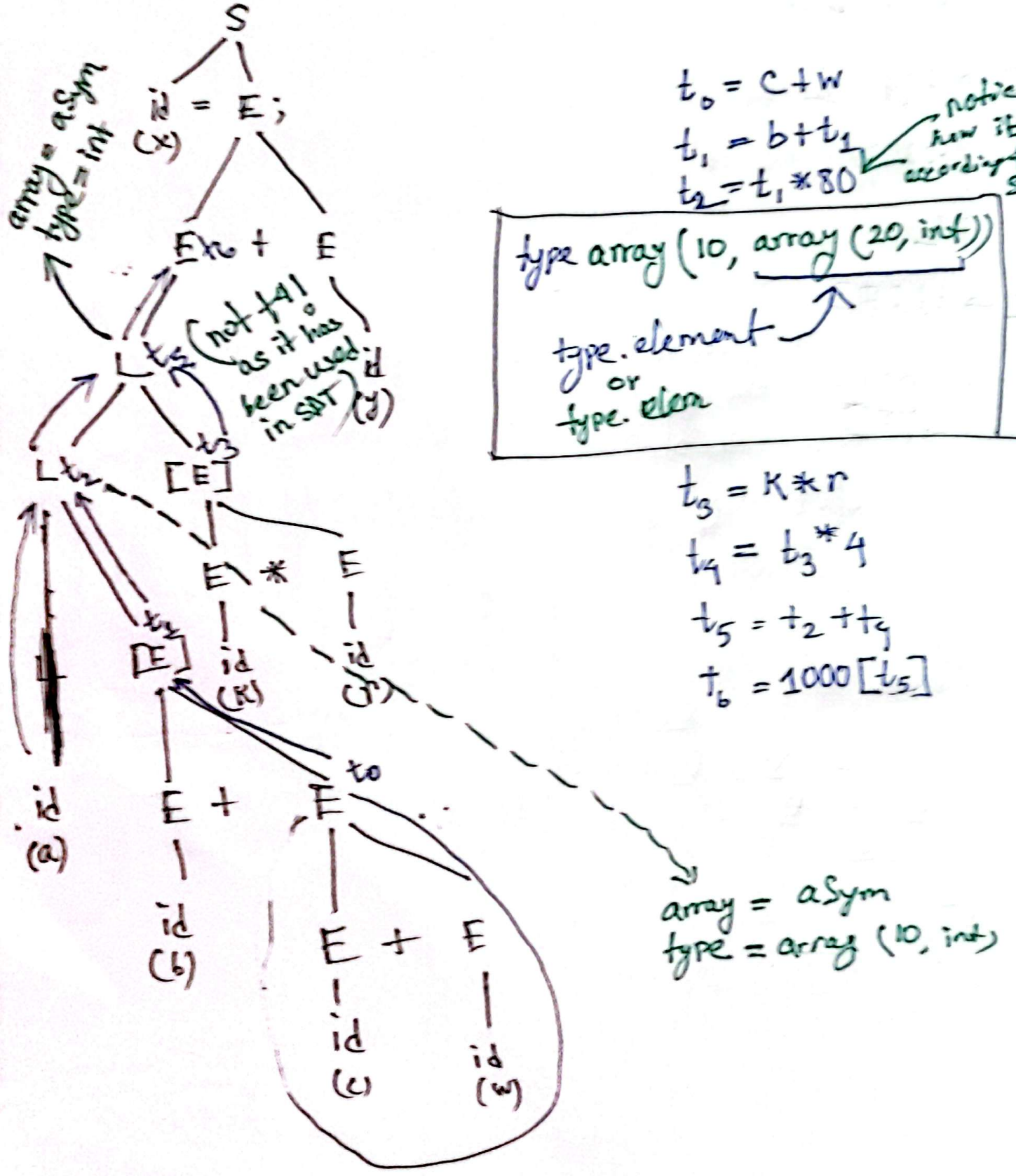
$\quad | \; L_1[E]$

Healthcare

Alexa
Enoxaparin USP

Scanned with CamScanner

$x = a[b+c+w][k*r] + y ;$



$t_0 = c + w$

$t_1 = b + t_1$

$t_2 = t_1 * 80$ ← notice how its according to SDT

type array (10, array (20, int))

type.element or type.elem

$t_3 = k * r$

$t_4 = t_3 * 4$

$t_5 = t_2 + t_4$

$t_6 = 1000[t_5]$

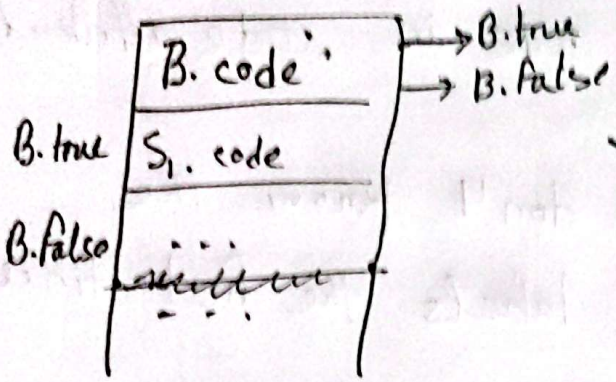
array = aSym

type = array (10, int)

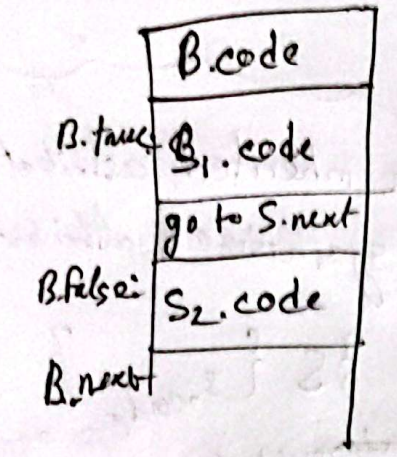
Scanned with CamScanner

25 Aug 2025

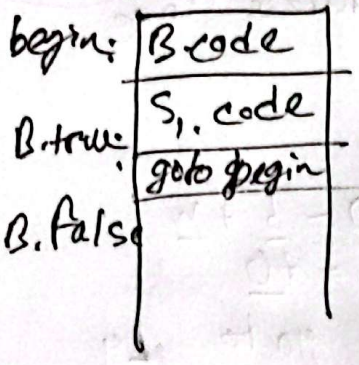
Control Flow




```
if (B) {
    S1
}
```




```
if (B) {
    S1
} else {
    S2
}
```




```
while (B) {
    S1
}
```

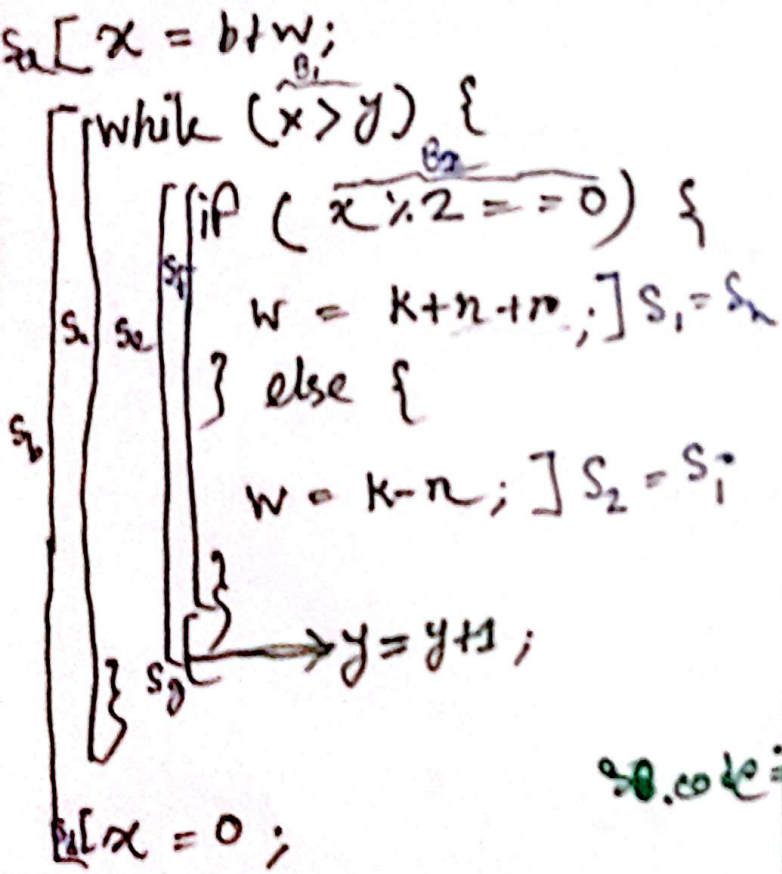
Page number for grammar : 406 (PDF page)

Healthcare
Alexa
Enoxaparin USP
Scanned with CamScanner

Grammar page number: 425 (PDF Page)

↳ Why are we storing code in .code attribute for later? ↴

Because we don't know the location of labels the first time



In the first

*Labels are inherited attributes

*Code is synthesized attributes

P → { } S { S.code = assign.code }
S.next = newlabel()

Three Address Code

S.code = assign1.code | B1.code | B2.code | E3.code | S1.code

E1.addr: t0

E1.code: t0 = b + w

assign1.code: t0 = b + w
x = t0

B1.code: if x > y go to L4

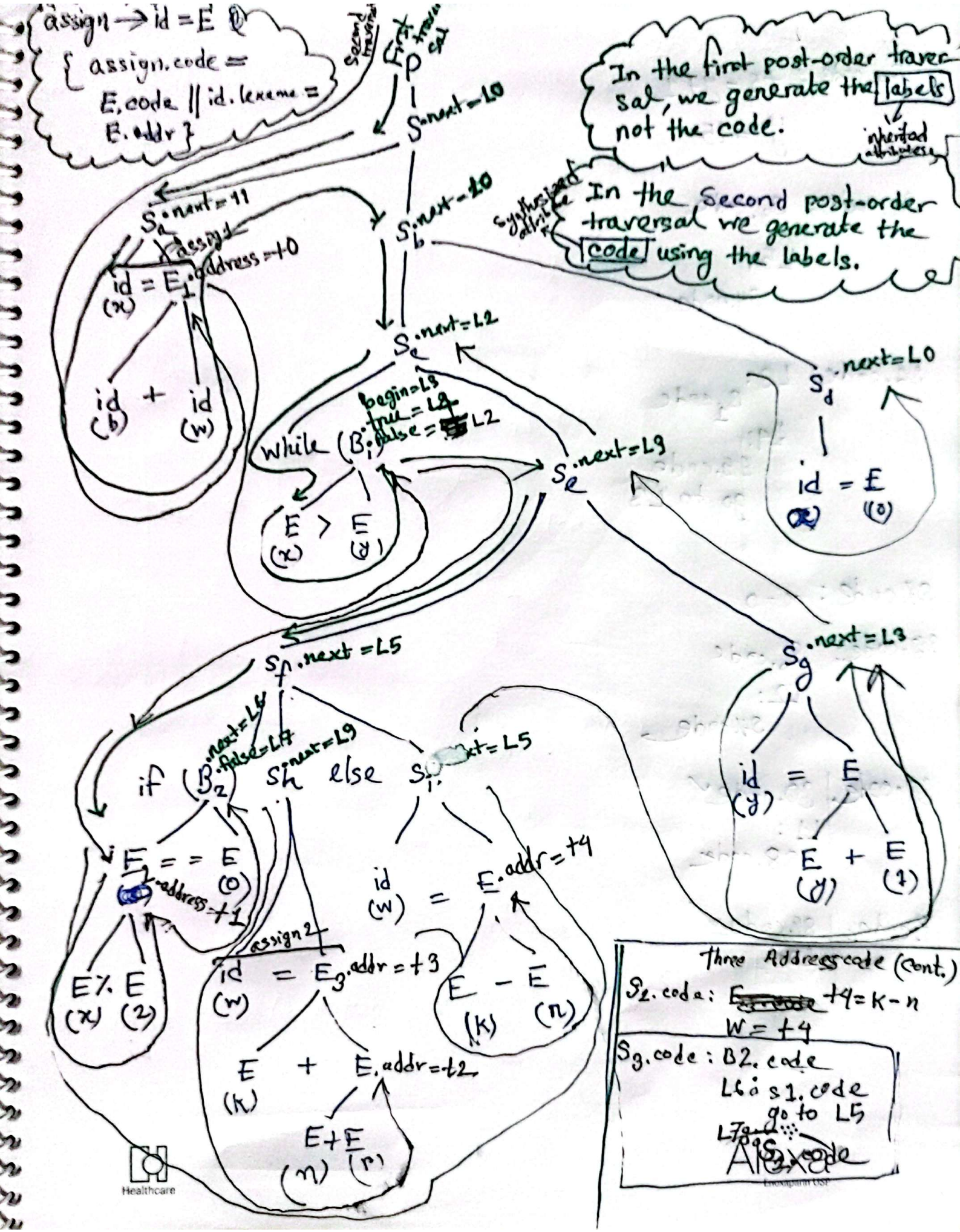
E2.code: t1 = x % 2

B2.code: t1 = x % 2
if t1 == 0 go to L6
go to L7

E3.code: t2 = n + m
t3 = k + t2

S1.code: E3.code
w = t3

Scanned with CamScanner



Scanned with CamScanner

$S_4.code$: $t5 = y + 1$
$y = t5$

$S_5.code$: $S_3.code$
L5:
$S_4.code$

$S_6.code$: L3:
$B_3.code$
L4:
$S_5.code$
go to L3

$S_7.code$: $x = 0$

$S_8.code$: $S_6.code$
L2:
$S_7.code$

$S_9.code$ | $S_0.code$ ✓
L1:
$S_8.code$

$P.code$ | $S_9.code$ ✓
L0:

Scanned with CamScanner

t0 = b+w
x = t0
L6

t0 = b+w
x = t0
L1:
L3: if x > y go to L9
go to L2
L4:

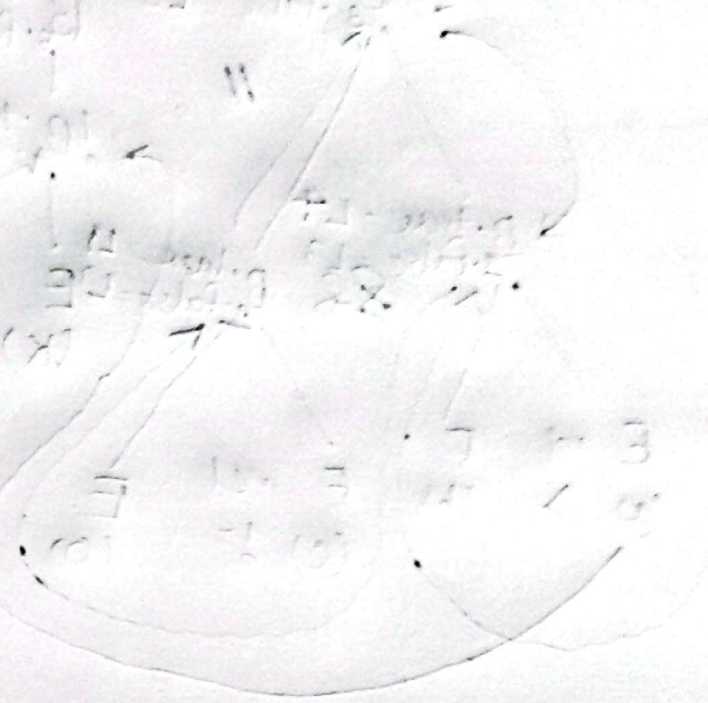
Healthcare
Alexa
Enoxaparin USP

Scanned with CamScanner

Sept 1, 2025

Logical connective: a>b && b!=0 (short circuiting condition code)

Code generation rules for boolean conditions

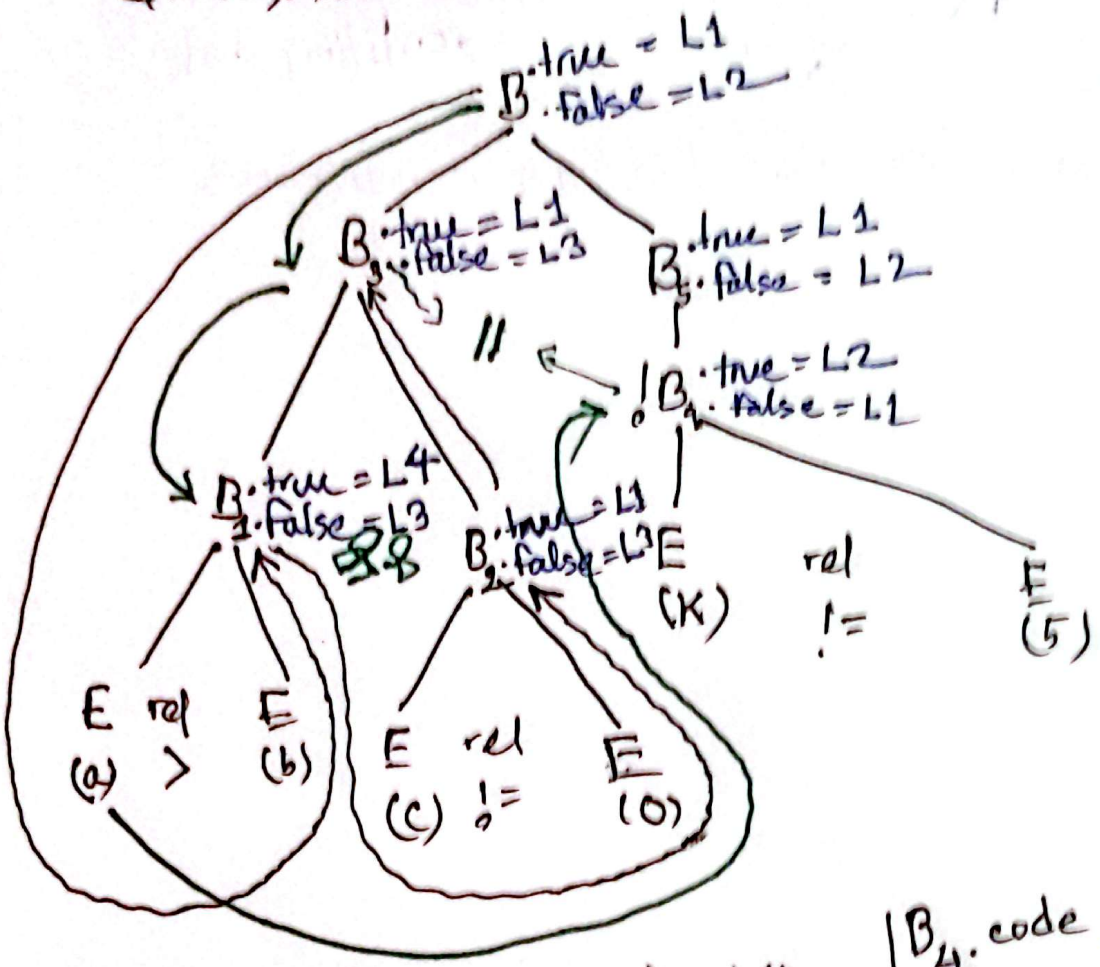


Healthcare

Alexa
Enoxaparin USP

Scanned with CamScanner

((a>b && c != 0) || !(K > 5))



B_1.code: if a>b go to L4
go to L3

B_2.code: if c!=0 go to L1
go to L3

B_3.code if a>b goto L4
go to L2

L4: if c!=0 go to L1
go to L3

B_4.code: if K>5 goto L2
go to L1

B_5.code: B_4.code

B_6.code: B_3.code
L3:
B_4.code

Scanned with CamScanner

```
if a>b go to L4
 go to L3
L4: if c1=0 go to L1
    go to L3
L3: if K>5 go to L2
    go to L1
```

Practice Problems

* Generate the code for

```
if (a[K+W+r+n][2+b[i]] >= 10 && f(x+y)>r){
  K=K+1 ;
}
```

Scanned with CamScanner